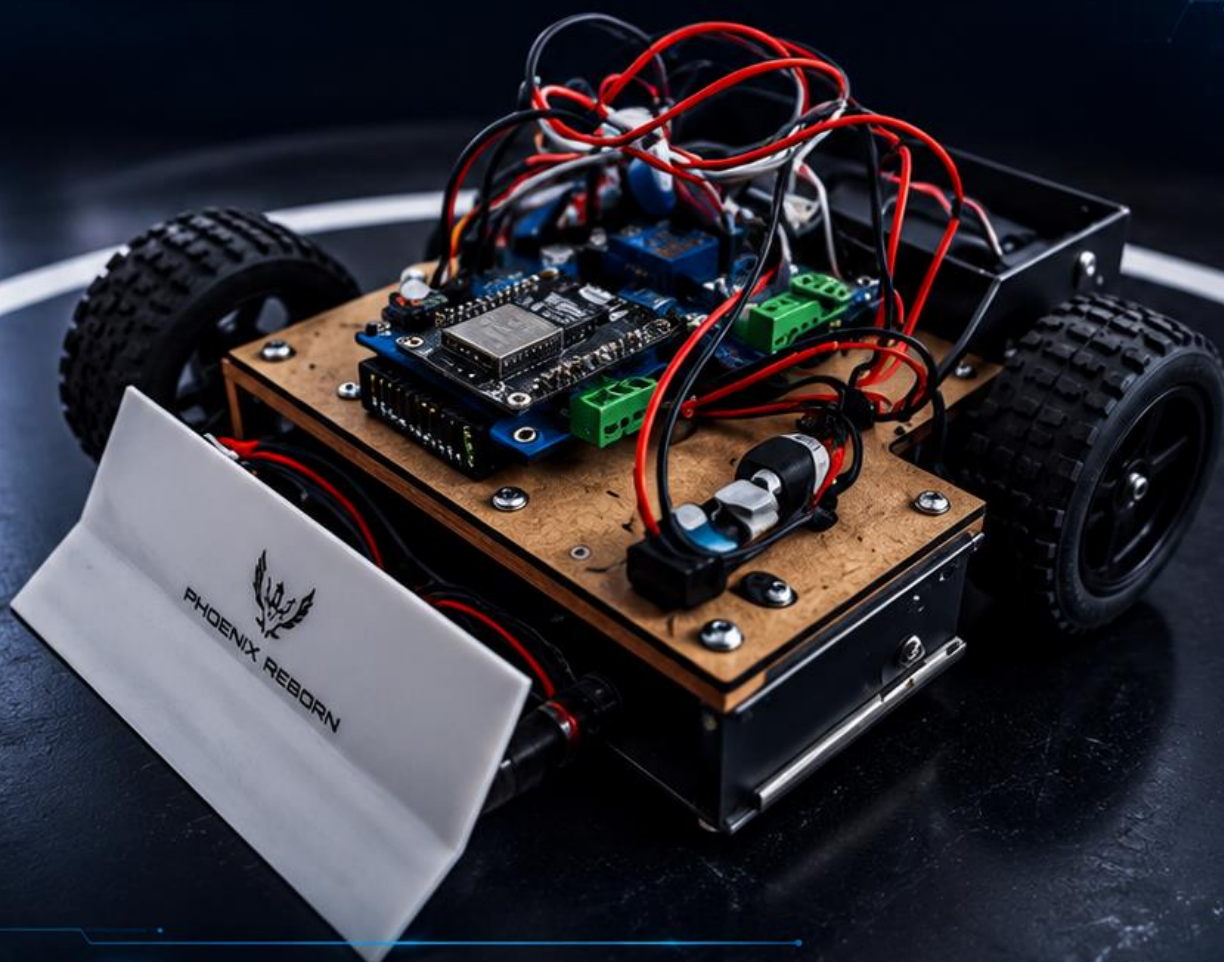




PHOENIX REBORN

ROBOT DE SUMO

MANUAL DE CONSTRUCCIÓN Y OPERACIÓN



ASESORA:

Lic. Ileana Haydee Resendiz lopez

Manual del Robot de Sumo

Componentes principales:

- ESP32 NodeMCU-32S
- Placa de expansión ESP32
- 2 Puentes H BTS7960
- 2 Step Down LM2596
- 4 Motores JGA25-370
- 4 Llantas con adaptadores hexagonales de 12 mm
- Batería LiPo 11.1 V 2200 mAh

1. Introducción

Este manual describe el funcionamiento, ensamblaje y mantenimiento de un robot de sumo diseñado para competencias de empuje. El sistema utiliza una ESP32 como controlador principal y motores de corriente continua accionados mediante puentes H BTS7960.

2. Descripción de los componentes

ESP32 NodeMCU-32S: Microcontrolador principal encargado del control y procesamiento.

Placa de expansión ESP32: Facilita las conexiones eléctricas y la distribución de energía.

Puentes H BTS7960: Permiten controlar la velocidad y dirección de los motores.

Step Down LM2596: Reducen el voltaje de la batería a niveles adecuados para la electrónica.

Motores JGA25-370: Proporcionan la fuerza de tracción necesaria para el combate.

Llantas con adaptadores de 12 mm: Transmiten la potencia de los motores al suelo.

Batería LiPo 11.1 V 2200 mAh: Fuente principal de alimentación del robot.

3. Ensamblaje general

1. Fijar los cuatro motores al chasis.
2. Instalar las llantas con los adaptadores hexagonales.
3. Montar la ESP32 y la placa de expansión.
4. Conectar los BTS7960 a los motores.
5. Ajustar los LM2596 al voltaje requerido.
6. Conectar la batería LiPo respetando la polaridad.

4. Alimentación eléctrica

La batería LiPo de 11.1 V alimenta el sistema. Los módulos LM2596 pueden utilizarse para suministrar voltajes adecuados a la electrónica de control y sensores.

5. Funcionamiento

La ESP32 ejecuta el programa principal y envía señales a los BTS7960 para controlar la velocidad y dirección de cada lado del robot. Esto permite avanzar, retroceder y girar.

6. Mantenimiento

Verificar las conexiones, el estado de las llantas, la carga de la batería y la temperatura de los controladores BTS7960 durante las pruebas.

7. Medidas de seguridad

No perforar ni sobrecargar la batería LiPo. Revisar la polaridad antes de energizar el robot y desconectar la batería cuando no esté en uso.

CODIGO PARA PROBAR EL ROBOT

```
#include "BluetoothSerial.h"
```

```
BluetoothSerial SerialBT;
```

```
// BTS IZQUIERDO
```

```
#define RPWM_L 25
```

```
#define LPWM_L 26
```

```
// BTS DERECHO
```

```
#define RPWM_R 27
```

```
#define LPWM_R 14
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    SerialBT.begin("PhoenixReborn");
```

```
    pinMode(RPWM_L, OUTPUT);
```

```
    pinMode(LPWM_L, OUTPUT);
```

```
    pinMode(RPWM_R, OUTPUT);
```

```
    pinMode(LPWM_R, OUTPUT);
```

```
    detener();
```

```
}
```

```
void loop() {
```

```
    if (SerialBT.available()) {
```

```
        char comando = SerialBT.read();
```

```
        switch (comando) {
```

```
case 'F':
```

```
    adelante();
```

```
    break;
```

```
case 'B':
```

```
    atras();
```

```
    break;
```

```
case 'L':
```

```
    izquierda();
```

```
    break;
```

```
case 'R':
```

```
    derecha();
```

```
    break;
```

```
case 'S':
```

```
    detener();
```

```
    break;
```

```
}
```

```
}
```

```
}
```

```
void adelante() {
```

```
digitalWrite(RPWM_L, HIGH);  
digitalWrite(LPWM_L, LOW);
```

```
digitalWrite(RPWM_R, HIGH);  
digitalWrite(LPWM_R, LOW);
```

```
}
```

```
void atras() {
```

```
digitalWrite(RPWM_L, LOW);  
digitalWrite(LPWM_L, HIGH);
```

```
digitalWrite(RPWM_R, LOW);  
digitalWrite(LPWM_R, HIGH);
```

```
}
```

```
void izquierda() {
```

```
digitalWrite(RPWM_L, LOW);  
digitalWrite(LPWM_L, HIGH);
```

```
digitalWrite(RPWM_R, HIGH);  
digitalWrite(LPWM_R, LOW);
```

```
}
```

```
void derecha() {  
  
    digitalWrite(RPWM_L, HIGH);  
    digitalWrite(LPWM_L, LOW);  
  
    digitalWrite(RPWM_R, LOW);  
    digitalWrite(LPWM_R, HIGH);  
}
```

```
void detener() {  
  
    digitalWrite(RPWM_L, LOW);  
    digitalWrite(LPWM_L, LOW);  
  
    digitalWrite(RPWM_R, LOW);  
    digitalWrite(LPWM_R, LOW);  
}
```